Transformer

Теоретический семинар

Область применения

Transformer предложен в 2017 году для машинного перевода. Механизм внимания применим к задачам, представимым как последовательность токенов с позициями.

Область	**Примеры моделей и задач**
NLP	машинный перевод, языковое моделирование, классификация текстов, чтение с пониманием, суммаризация
Речь	распознавание речи (Speech-Transformer), обучение речевых представлений (wav2vec 2.0)
Зрение и мультимодальность	Vision Transformer (ViT), DINOv2, CLIP, BLIP/BLIP-2, Segment Anything
Кросс-модальные модели	Gemini 1.5, SeamlessM4T
Научные области	предсказание структуры белков, белок-белковых взаимодействий, мутаций

Проблемы RNN

Ограничение	**Формулировка**
Последовательные вычисления	Вычисление на шаге t зависит от h_{t-1}. Параллелизация внутри одного примера невозможна. На длинных последовательностях ограничения памяти сжимают batch size.
Длина пути для дальних зависимостей	Сигнал от позиции 1 до позиции n проходит $O(n)$ шагов. Градиент проходит через n последовательных умножений. Обучение дальним зависимостям затруднено.
Длина пути в свёрточных сетях	ConvS2S — $O(n)$, ByteNet — $O(\log n)$. Число операций для связи двух произвольных позиций растёт с расстоянием.

Сравнение архитектур

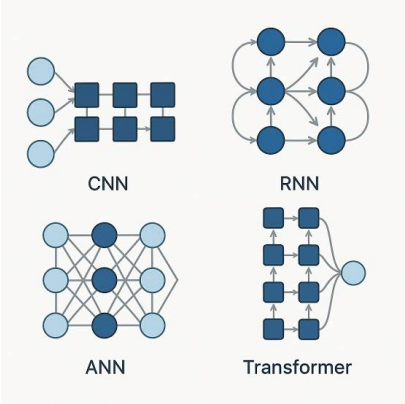


Рис.: https://medium.com/@savindufernando/comparing-cnn-rnn-and-transformer-models-in-simple-terms-a859fe42a299

Определение: ConvS2S

ConvS2S (Convolutional Sequence to Sequence) — архитектура sequence-to-sequence на основе свёрточных сетей, предложенная Gehring et al. (2017). Encoder и decoder состоят из стеков свёрточных слоёв; вычисления для всех позиций выполняются параллельно.

Ключевое свойство. Число операций для связи двух произвольных позиций растёт линейно с расстоянием: $O(n)$.

Параметры в оригинальной работе. 15 слоёв в encoder и decoder; максимальный размер контекста; BLEU 25.16 на WMT'14 En-De.

Определение: ByteNet

ByteNet (Neural Machine Translation in Linear Time) — архитектура sequence-to-sequence на основе свёрточных сетей с дилатацией, предложенная Kalchbrenner et al. (2016). Encoder и decoder состоят из стеков дилатированных свёрточных слоёв; вычисления для всех позиций выполняются параллельно.

Ключевое свойство. Благодаря дилатации число операций для связи двух произвольных позиций растёт логарифмически с расстоянием: $O(\log n)$.

Параметры в оригинальной работе. BLEU 23.75 на WMT'14 En-De.

Определение: Extended Neural GPU

Extended Neural GPU — архитектура на основе свёрточных сетей, предложенная Kaiser & Sutskever (2016). Относится к тому же классу моделей, что ConvS2S и ByteNet: использует свёртки как базовый строительный блок и вычисляет скрытые представления для всех позиций параллельно.

Общая цель с Transformer. Сокращение последовательных вычислений за счёт параллельной обработки всех входных и выходных позиций.

Проблемы RNN: самостоятельная работа

Вопрос. Прочитайте аннотацию и Section 1 оригинальной статьи. Найдите формулировку «фундаментального ограничения» (fundamental constraint) RNN. Ответьте одной фразой из оригинала.

Ключевые решения Transformer

Решение	**Содержание**
Отказ от рекуррентности и свёрток	Зависимости между входом и выходом устанавливаются только механизмом внимания.
Self-Attention	Число операций между любыми двумя позициями константно $O(1)$. Эффективное разрешение снижается; компенсируется multi-head attention.
Параллелизация	Все позиции вычисляются одновременно. В оригинале обучение до SOTA заняло 12 часов на 8 GPU P100.
Позиционное кодирование	Порядок, утраченный после удаления рекуррентности, восстанавливается добавлением позиционного вектора к эмбеддингу.

Обзор компонентов

№	Компонент
1	Стек encoder-decoder
2	Scaled Dot-Product Attention
3	Multi-Head Attention
4	Позиционное кодирование
5	Residual connections и LayerNorm
6	Маскирование

Компонент 1. Стек encoder-decoder

Определение. Encoder содержит $N = 6$ одинаковых слоёв. Каждый слой включает два под-слоя: multi-head self-attention и position-wise feed-forward network. Decoder также содержит $N = 6$ слоёв, в каждом на один под-слой больше — encoder-decoder attention. Все под-слои обёрнуты в residual connection и LayerNorm. Выходная размерность всех под-слоёв унифицирована: $d_{model} = 512$.

Стек encoder и decoder

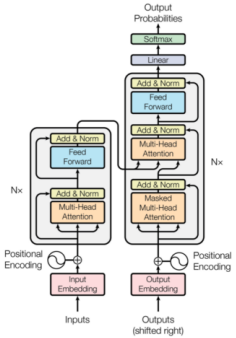


Рис.: Figure 1 оригинальной статьи. Слева — encoder: $N = 6$ слоёв, каждый с двумя под-слоями (multi-head self-attention, feed-forward). Справа — decoder: $N = 6$ слоёв, каждый с тремя под-слоями (masked self-attention, encoder-decoder attention, feed-forward). Стрелки показывают поток данных и точки residual connection + LayerNorm.

Компонент 1. Стек encoder-decoder: задание

Самостоятельная работа. По Figure 1 оригинальной статьи:

1. Сколько под-слоёв в encoder и decoder?
2. Как называется дополнительный под-слой decoder?
3. Откуда берутся Q, K, V для этого под-слоя?

Q, K, V в encoder-decoder attention

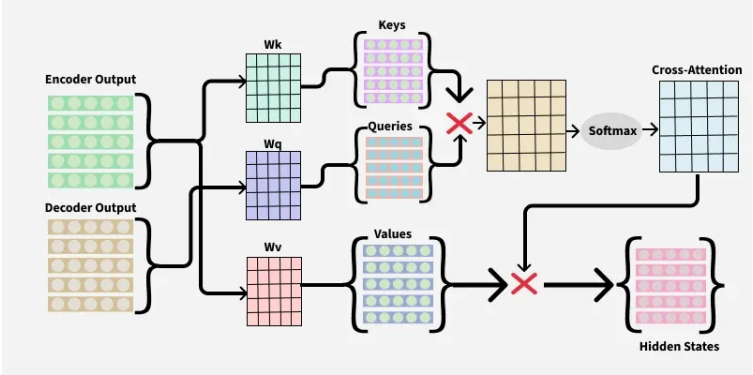


Рис.: Encoder-decoder attention. Q приходит из выхода предыдущего слоя decoder; K и V — из выхода encoder. Каждый слой decoder «запрашивает» релевантные позиции во входной последовательности.

Компонент 2. Scaled Dot-Product Attention

Определение. Функция внимания отображает query и набор пар key-value в выход. Выход — взвешенная сумма value; веса определяются совместимостью query с соответствующим key:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Scaled Dot-Product Attention

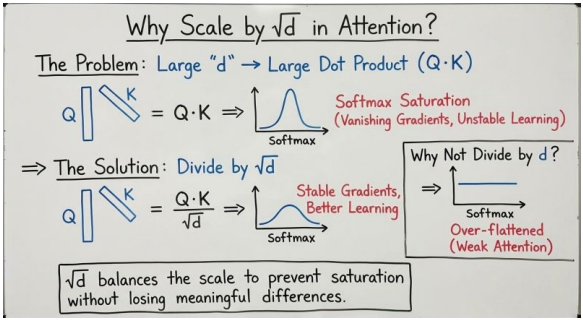


Рис.: https://www.linkedin.com/posts/iamhimanshu0_in-scaled-dot-product-attention-we-divide-activity-7407439635209134080-WRBv

Матричное умножение QK^T, масштабирование на $1/\sqrt{d_k}$, softmax по строкам, умножение на V. Для каждой позиции получается распределение весов по всем key.

Определение: additive attention

Additive attention (Bahdanau attention) — функция внимания, в которой совместимость query и key вычисляется через полносвязную сеть с одним скрытым слоем, а не через скалярное произведение.

Сравнение с dot-product attention. Теоретическая сложность сопоставима. На практике dot-product attention быстрее и экономнее по памяти за счёт оптимизированных матричных операций.

Компонент 2. Scaled Dot-Product Attention: задание

Самостоятельная работа. Прочитайте Section 3.2.1 оригинала и ответьте на три вопроса:

1. Зачем делить на $\sqrt{d_k}$?
2. Почему dot-product attention на практике эффективнее additive attention при сопоставимой теоретической сложности?
3. Что произойдёт при больших d_k, если не масштабировать?

Компонент 3. Multi-Head Attention

Определение. Q, K, V проецируются в $h = 8$ подпространств меньшей размерности. Внимание выполняется параллельно в каждом подпространстве. Результаты конкатенируются и проецируются снова.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V)$$

Multi-Head Attention

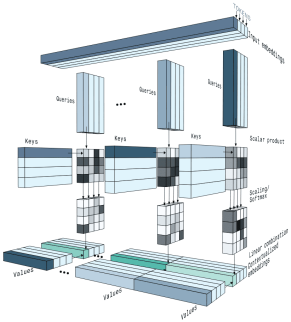


Рис.: https://theaisummer.com/self-attention/

Q, K, V линейно проецируются $h = 8$ раз; каждая голова выполняет scaled dot-product attention независимо; результаты конкатенируются; финальная линейная проекция W^O возвращает размерность d_{model}.

Компонент 3. Multi-Head Attention: задание

Самостоятельная работа. Прочитайте Section 3.2.2 оригинала. Параметры оригинала: $h = 8$, $d_{model} = 512$, $d_k = d_v = d_{model}/h = 64$.

Вопрос. Поскольку размерность каждого head снизилась, вычислительная стоимость по сравнению с single-head при $d_{model} = 512$ увеличилась или уменьшилась? Почему?

Определение: permutation invariant

Инвариантность к перестановке (permutation invariant) — свойство функции, при котором перестановка элементов входа не меняет выход.

В контексте self-attention. Без позиционного кодирования self-attention не различает порядок токенов: перестановка токенов приводит к перестановке выходов, но не меняет их значения. Позиционное кодирование нарушает эту инвариантность, добавляя информацию о позиции.

Компонент 4. Позиционное кодирование

Определение. Self-attention инвариантен к перестановке входа (permutation invariant): перестановка токенов не меняет выход. Позиционное кодирование добавляет к эмбеддингу каждого токена вектор, зависящий от позиции. Оригинал использует синусы и косинусы:

$$PE_{(pos,\,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos,\,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Позиционные кодировки

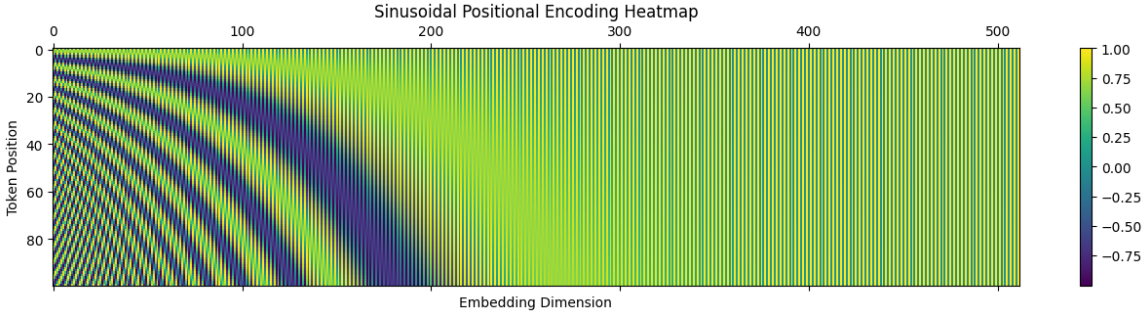
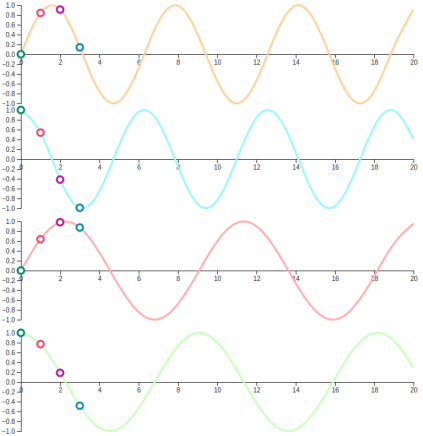


Рис.: https://www.ibm.com/think/topics/positional-encoding

Тепловая карта позиционных кодировок. По оси X — измерение эмбеддинга, по оси Y — позиция. Видны синусоидальные полосы с разной частотой.



p0	p1	p2	p3	
0.000	0.841	0.909	0.141	i=0
1.000	0.540	-0.416	-0.990	i=1
0.000	0.638	0.983	0.875	i=2
1.000	0.770	0.186	-0.484	i=3

Positional Encoding

$$PE_{(pos,2i)} = sin(\frac{pos}{10000^{2i/d_{\text{model}}}})$$

$$PE_{(pos,2i+1)} = cos(\frac{pos}{10000^{2i/d_{\text{model}}}})$$

Settings: d = 50

The value of each positional encoding depends on the *position* (*pos*) and *dimension* (*d*). We calculate result for every *index* (*i*) to get the whole vector.

Рис.: https://erdem.pl/2021/05/understanding-positional-encoding-in-transformers/

Компонент 4. Позиционное кодирование: задание

Самостоятельная работа. Прочитайте Section 3.5 оригинала и ответьте:

1. Почему не использовать целые числа 1, 2, 3, ... для кодирования позиции?
2. Какое ключевое преимущество синус-косинусной схемы (оригинал использует оборот «would allow the model to...»)?

Компонент 5. Residual connections и LayerNorm

Определение. Выход каждого под-слоя:

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

где x — вход под-слоя. Residual connection смягчает исчезновение градиентов в глубоких сетях. LayerNorm стабилизирует обучение и ускоряет сходимость.

Компонент 5. Residual connections и LayerNorm: задание

Самостоятельная работа. Почему оригинал требует, чтобы «all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{model} = 512$»?

Определение: auto-regressive

Авторегрессионная генерация (auto-regressive) — режим, при котором модель предсказывает следующий токен на основе всех предыдущих токенов. На шаге i модель имеет доступ только к токенам с индексами меньше i.

Следствие для обучения. Условия обучения должны совпадать с условиями инференса: модель не должна видеть будущие токены при предсказании текущего.

Компонент 6. Маскирование

Определение. Под-слой self-attention в decoder модифицирован: разрешён просмотр только текущей и предыдущих позиций. Реализация: до softmax оценки внимания для будущих позиций устанавливаются в $-\infty$; после softmax их веса становятся нулевыми. В сочетании со сдвигом выходных эмбеддингов на одну позицию это гарантирует, что предсказание для позиции i зависит только от известных выходов с индексами меньше i.

Схема: треугольная маска в матрице внимания

[Схема для объяснения: матрица внимания $seq_len \times seq_len$. Верхний правый треугольник (будущие позиции) заполнен $-\infty$; после softmax эти позиции дают нулевые веса. Нижний левый треугольник (прошлое и текущая позиция) содержит ненулевые веса.]

Компонент 6. Маскирование: задание

Самостоятельная работа. Прочитайте Section 3.1 (абзац про decoder) и Section 3.2.3 (или раздел про masked self-attention в туториале).

Вопрос. Почему маскирование нужно в decoder и не нужно в encoder?

Схема: encoder видит всё, decoder видит только прошлое

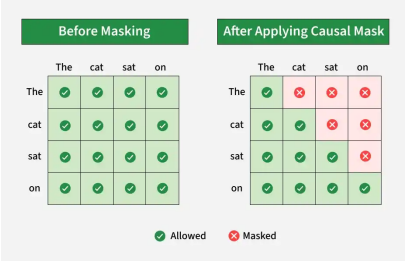


Рис.: https://www.geeksforgeeks.org/nlp/how-do-self-attention-masks-work/

Слева — encoder, self-attention без маски, каждая позиция видит все позиции входа. Справа — decoder, masked self-attention, каждая позиция видит только себя и предыдущие.

Источники

1. Jay Alammar, «The Illustrated Transformer» — визуализация self-attention и multi-head attention.
2. Harvard NLP, «The Annotated Transformer» — построчная реализация на PyTorch.
3. Section 3 оригинальной статьи — авторитетный источник ответов на все вопросы; при расхождениях с туториалами приоритет у оригинала.